

Tema 4

Seguridad en Redes

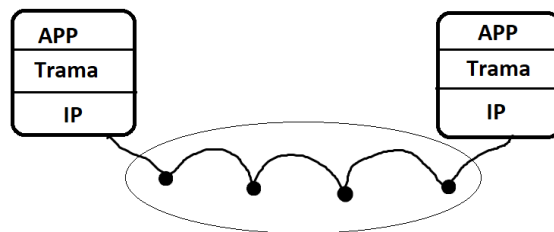
4.1. INTRODUCCIÓN

Recordemos: tenemos una arquitectura en capas donde la app se encapsula en tramas, las tramas en IPs, etc.

La seguridad es un problema a tratar.

La seguridad es un problema multidimensional → Afecta a varios aspectos.

¿Cuándo es una red segura? ¿Cuándo se garantiza? Una red de comunicaciones es segura cuando se garantizan todos los aspectos. No hay protocolos ni redes 100% seguros.



Los aspectos son:

- **Confidencialidad/privacidad:** El contenido de la información es comprensible sólo por entidades autorizadas.
- **Autenticación:** Las entidades son quien dicen ser. Garantía feaciente sin fisuras de la identidad → de que A es A.
- **Control de Accesos:** Los servicios están accesibles sólo a entidades autorizadas. Se trata de garantizar de forma feaciente (sin fisuras). No es tanto la información sino el servicio en sí (DHCP, RIP, SSH...) → Quiero que solo se puedan acceder a través de las entidades autorizadas.
- **No repudio o irrenunciabilidad:** el sistema impide la renuncia de la autoría de una determinada acción. Si no puede repudiar la autoría de algo, sería caótico.
- **Integridad:** Garantizar que el sistema detecta todas las alteraciones (intencionadas o no) de la información. Es imposible evitar las alteraciones pero sí que podemos detectarlas
- **Disponibilidad:** El sistema mantiene las prestaciones de los servicios con independencia de la demanda.

Cuando podemos garantizar todo esto, el sistema parece seguro. NUNCA AL 100%

¿En qué nivel se debe situar la seguridad? En TODOS... Hay que poner seguridad en todos los lados de la arquitectura por capas. El grado/nivel de seguridad lo fija el punto más débil.

Ataque de seguridad: Cualquier acción intencionada o no (errores) que menoscaba o pone en entredicho cualquiera de los aspectos de seguridad mencionados anteriormente.

Tipos de ataques:

- **Sniffing:** Ataques/vulneración de la confidencialidad. Escuchas a la información (husmear).
- **Spoofing:** Suplantación de la identidad de entidades. Ataques a la autenticación.
 - Pishing: Captura de información una vez hecho spoofing
- **Man in the middle:** hombre en medio. Ataques de interceptación
- **Distributed Denial Of Service (DDoS):** Denegación de servicio distribuido. Ataques a la disponibilidad. Por ejemplo, el Flooding (inundación)
- **Malware:** Software malintencionado: Troyanos, gusanos, spyware, backdoors, rookits, ransomware, keyloggers...

MECANISMOS DE SEGURIDAD

- De prevención: anticiparnos al ataque (intencionado o no). Nos centraremos en estos
 - Mecanismos de autenticación e identificación
 - Mecanismos de control de acceso.
 - Mecanismos de separación (física, temporal, lógica, criptográfica y fragmentación).
 - Mecanismos de seguridad en las comunicaciones (cifrado de la información).
- De detección: Poner herramientas de inspección
 - IDS (Intruder Detection System): Software de análisis. Hace monitorización del tráfico y detecta anomalías y eventos sospechosos.
- De recuperación: Una vez que hemos sufrido el ataque.
 - Copias de seguridad (backups)
 - Mecanismos de análisis forense: averiguar alcance, las actividades del intruso en el sistema y cómo entró.

HERRAMIENTAS PARA LA SEGURIDAD

- ★ Cifrado (simétrico y asimétrico)
- ★ Autenticación con clave secreta (reto-respuesta)
- ★ Intercambio de Diffie-Hellman (establecimiento de clave secreta)
- ★ Funciones Hash. Hash Message Authentication Code (HMAC).
- ★ Firma Digital.
- ★ Certificados digitales.

4.2. CIFRADO (SIMÉTRICO Y ASIMÉTRICO)

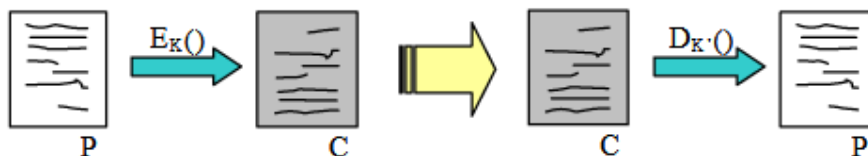
Se trata de realizar un intercambio de información sin que un tercero conozca/acceda a la información.

Está basado en la criptografía y es un procedimiento utilizado para garantizar la confidencialidad → Restringir el acceso a terceros no deseados.

Se trata de transformar el texto plano/claro P a través de un algoritmo o mecanismo tal que este proceso es irreversible a menos que conozcamos una información: la clave. Como resultado, obtendremos el texto cifrado, C . $P \rightarrow E_K(P) = C$ donde K es la clave y E es un algoritmo

Si no conoces la clave, no puedes cifrar/descifrar la información. Se basa en la existencia de un algoritmo de cifrado/descifrado, normalmente conocido $E_K()$ y $D_{K'}()$. La dificultad reside en la existencia de unas claves de cifrado (K) y descifrado (K') desconocidas.

El descifrado se trata de, utilizando una clave recuperamos la información inicial. $D_{K'}(C) = P$ donde K' es la clave y D el algoritmo.



Tipos de cifrado:

- ★ Cifrado simétrico o con clave secreta: $K = K'$.
 - Computacionalmente son más fáciles (menos costosos)
- ★ Cifrado asimétrico o con clave pública/privada. K_{PUB}/K_{PRIV} .
 - Más complicados (más costosos).
 - Más robustos/fuertes/difíciles de “vulnerar”.

CIFRADO SIMÉTRICO, ALGORITMOS DE CLAVE SECRETA:

Existe una sola clave para cifrar y descifrar ($k=k'$)

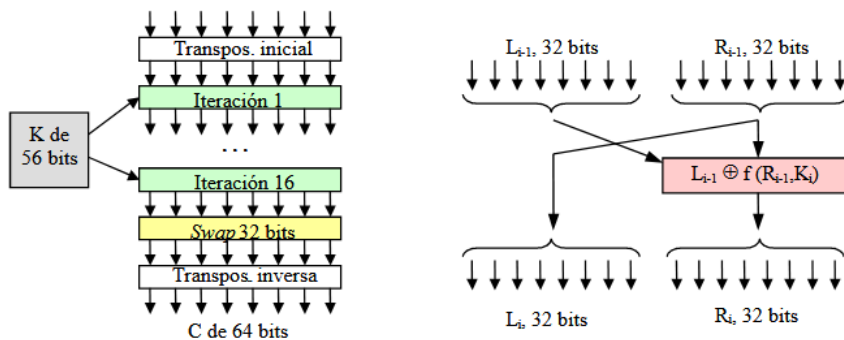
DES ("Data Encryption Standard"): Estándar que se acordó como norma. Fue especificado en el 75 por IBM y propuso un algoritmo de cifrado simétrico. Este algoritmo es considerado no seguro.

Cuando se encuentra alguna metodología que reduzca el espacio de búsqueda por fuerza bruta en cualquier orden de magnitud, es no seguro. P.e 56 bits $\rightarrow 2^{56}$ claves $\rightarrow$ voy probando una a una. Si alguien ve la forma de reducir de 2^{56} a 2^{45} , es no seguro.

Pese a no ser seguro, se sigue usando.

DES coje el texto plano. Los agrupa en bloques de 64 bits. Hace operaciones con la clave. Sale una cadena de 64 bits.

Para descifrar, DES aplica la clave al revés. Se cogen los subconjuntos de la clave en orden inverso. Por eso se llama cifrado simétrico.



DES: Es un esquema de sustitución monoalfabético. Entra un símbolo y sale otro.

a	b	c		y	z
↓	↓	↓		↓	↓
m	x	e		d	r

Esto es lo que hace DES

Los esquemas de sustitución monoalfabético no son robustos.

En el ejemplo anterior hay $26 \cdot 25 \cdot 24 \cdot \dots \cdot 2 \cdot 1 = 26!$ Sustituciones posibles.

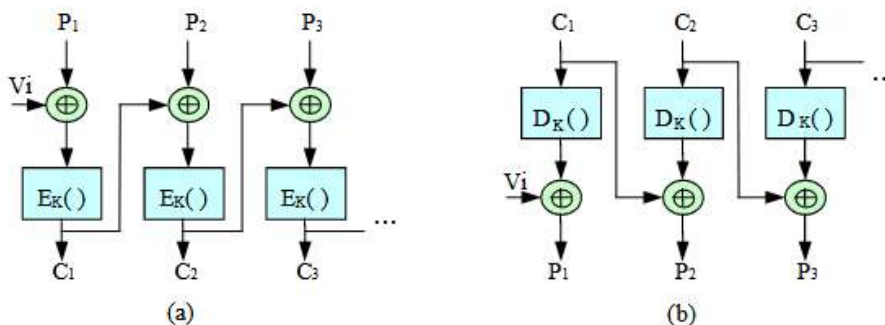
¿Cómo se reduce la complejidad para reducir el espacio de opciones posibles? Adopto la hipótesis de que es castellano.

La letra más posible por estadística es la "e" en el castellano. Cojo el texto cifrado y veo el que más se repite. Por ejemplo la "c". Tengo ese conocimiento a priori. Ahora son 25!. Porque supongo que esa letra es la "e"

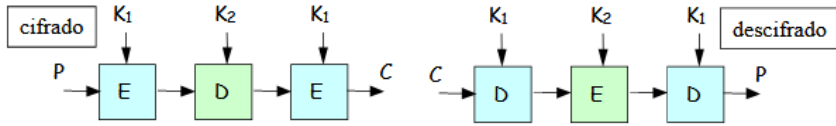
Podríamos aplicar la misma hipótesis con la segunda más utilizada y así sucesivamente.

¿Cómo mejorar esa debilidad?

- DES Encadenado: Para evitar que DES sea un algoritmo de sustitución. Intentar que el símbolo de salida no solo dependa del símbolo inicial sino de la historia pasada.



DES Doble: Pasar de 56 bits a 112 bits. Dos cifrados consecutivos. Cifro con una clave y vuelvo a cifrar → más robusto.

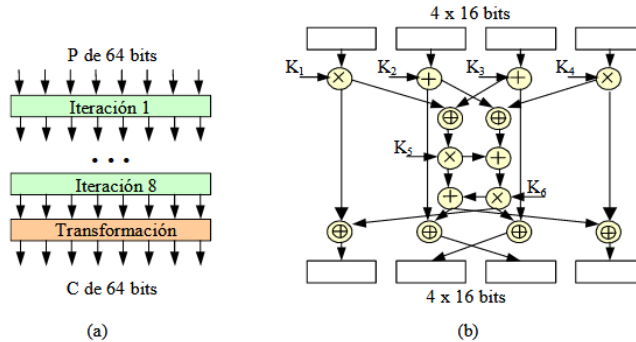


DES Triple: 168 BITS. Cifrado, descifrado y cifrado con tres claves.

Existen alternativas seguras a DES:

IDEA (“International Data Encryption Algorithm”)

Simétrico: misma clave para cifrar y para descifrar. Claves de 128 bits. Opera en tiempo real



Los algoritmos de cifrado simétrico son muy rápidos, eficientes y tienen muy poca carga. Pero no son tan robustos como los de cifrado asimétrico:

CIFRADO ASIMÉTRICO, ALGORITMOS DE CLAVE PÚBLICA/PRIVADA

Consiste en la asociación para cada usuario/entidad de dos claves: una clave pública y otra clave privada.

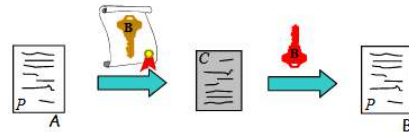
$A \rightarrow \{K_{\text{PUB}} / K_{\text{PRIV}}\}$

Conocida la clave pública es computacionalmente imposible averiguar la privada.

Caves diferentes para cifrar y descifrar:

Cifrar $\rightarrow C = E_{K_{\text{pub-B}}}(P)$

Descifrar $\rightarrow D_{K_{\text{priv-B}}}(C) = D_{K_{\text{priv-B}}}(E_{K_{\text{pub-B}}}(P)) = P$



Si A quiere enviarle un texto a B cifrado, A lo cifra con la clave pública del destino (B), Como la pública es conocida por todo el mundo, no hay problema. B es el único que conoce la clave privada que aplicada a la información cifrada con la clave pública, recupera la información sin cifrar (la descifra). Solo lo puede hacer B.

$A \rightarrow B$

$K_{\text{PUB-B}}(P) \rightarrow K_{\text{PRIV-B}}(K_{\text{PUB-B}}(P)) = P$

¿Y si enviamos $C = E_{K_{priv-A}}(P)$? ¿Sirve como autenticación?

$A \rightarrow B$

$K_{PRIV-A} \rightarrow K_{PUB-A}(K_{PRIV-A}(P)) = P$

De esta forma podemos autenticar al origen Sabemos que A es A

Para ello tenemos que garantizar de forma fehaciente que A está asociado a su clave pública ($A \rightarrow K_{PUB-A} \rightarrow$ CERTIFICADO DIGITAL (asociación fehaciente de tu identidad con tu clave pública))

Pero... ¿cómo se generan las llaves públicas y privadas?

RSA

Elegimos p y q primos grandes ($> 10^{1000}$).

Construimos dos funciones: $n=(p*q)$ y $z=(p-1)*(q-1) \rightarrow$ función de Euler

Elegimos un d primo respecto a z.

Calculamos e tal que $e*d \bmod z = 1$ (algoritmo de Euclides)

$K_{pub} = (e,n)$ y $K_{priv} = (d,n)$, de modo que:

$$C = p^e \bmod n$$

$$P = C^d \bmod n$$

Ejemplo:

■ $p = 3, q = 11$

■ $n = p.q = 33, z = (p-1)(q-1) = 20 = 5 \times 2 \times 2$

■ $d = 7$, primo respecto de z

■ $e = 3, e \times d \bmod z = 1$

■ $K_{pub}=(3,33)$ y $K_{pri}=(7,33)$

En resumen, tenemos dos tipos de cifrado:

- $K_{secreta} \rightarrow$ eficaz computacionalmente y utilizada mucho para protocolos
- $K_{pub}/K_{priv} \rightarrow$ menos eficaz pero más robusto.

4.3. AUTENTICACIÓN CON CLAVE SECRETA (RETO-RESPUESTA)

Autenticar consiste en verificar de forma fehaciente (sin fisuras) la identidad de las entidades involucradas. "A es quien dice ser". A puede ser un usuario, aplicación, puerto, IP, MAC...

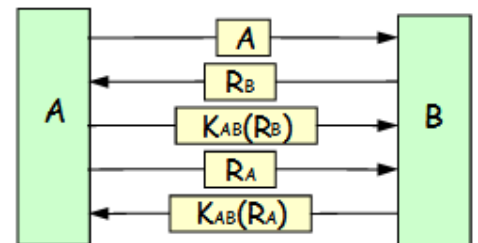
Autenticar no es [login/password] $\rightarrow$ es identificar. No es autenticar porque no es de forma fehaciente y segura. $\rightarrow$ Por repetición se puede adivinar. No es nada seguro el password.

$A \leftrightarrow K_{PUB_A}$ (A está vinculada con su clave pública) $\rightarrow$ Certificado digital (lo veremos más adelante).

Hipótesis: $\exists K_{AB} \rightarrow$ secreta R es random.

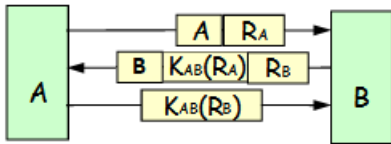
Suponemos que existe una clave secreta conocida solo por A y B que me va a ayudar por un protocolo que A sepa que B es B. $\rightarrow$ Reto respuesta. Es un procedimiento de autenticación y cifrado basado en clave respuesta.

A le dice a B que es A. B reenvía un número aleatorio y le reta a que cifre X mensaje que solo él conoce. A cifra y se lo pasa a B. También le pasa otro número aleatorio como reto para que B cifre. B cifra y compara con lo recibido de A. Además, envía el mensaje cifrado a A (el reto que le propuso al final)



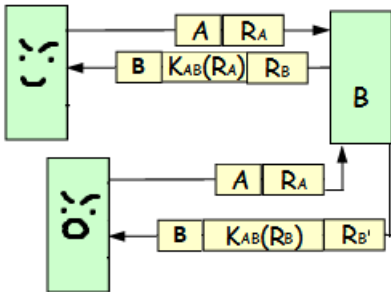
Aquí no hay repetición. Está protegida porque hay un rango de tiempo.

¿Es posible simplificar el procedimiento a 3 mensajes? Si:



A le dice a B que es A y le manda el reto
 B responde diciendo que Es B, le manda el reto cifrado y le deja su reto
 A le responde con el reto cifrado

¿Esto es vulnerable a un ataque por Reflexión? Si.



Coge el reto cifrado y cierra la sesión engañando a B.

Solución: Usar espacios de retos disjuntos. Es decir, que los random sean disjuntos.

-----> Retos pares
 <----- Retos impares

Recomendación: Usar nonces: Solo se puede usar una vez → Información de un solo uso.
 Si introduzco en mis intercambios los nonces, evito ataques por repetición.

Los timestamps son un buen candidato (marca de tiempo).

4.4. INTERCAMBIO DE DIFFIE-HELLMAN (ESTABLECIMIENTO DE CLAVE SECRETA)

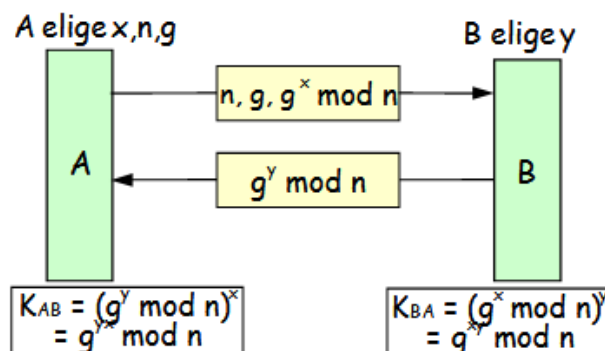
¿Seríamos capaces de intercambiar información a voces sin que nadie se entere de la clave secreta? Elijo 3 números: x , n y g

Envío n , g y $g^x \mod n$ a B

B elige un número → Coge el tercer número y lo eleva a su número elegido (y) → Clave secreta
 → Devuelve $g^y \mod n$

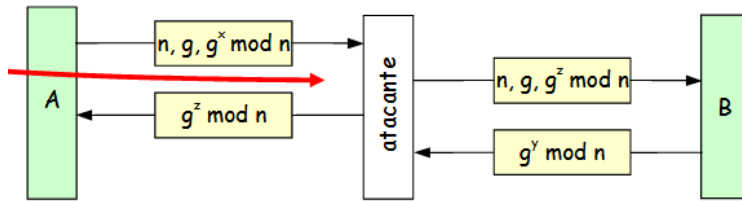
Yo cojo ese número y lo elevo a x . → Clave secreta

El intercambio de Diffie-Hellman: Permite establecer una clave secreta entre dos entidades a través de un canal no seguro.



No se puede averiguar, es computacionalmente imposible. La gente sabe los dos primeros números, pero no el último.

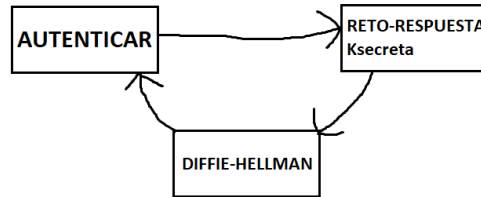
Debilidad: Nos permite establecer una clave secreta con el otro extremo, pero es susceptible a “Ataques man-in-the-middle:



Alguien se interpone en medio → Posible ataque de spoofing.

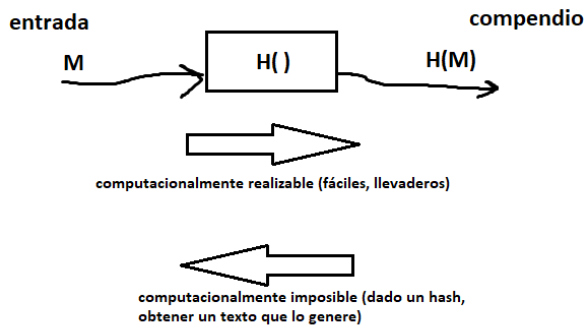
Solución: Funciona bien si tenemos autenticado al otro extremo. Si no hay autenticación previa, este intercambio se ve comprometido

Para autenticar hemos visto que necesitamos una clave secreta por reto-respuesta, pero para ella necesitamos Diffie-Hellman. Pero además para Diffie-Hellman necesitamos autenticación. Entramos en un bucle pero veremos que hay alternativas.



4.5. FUNCIONES HASH. HASH MESSAGE AUTHENTICATION CODE (HMAC)

Transformación de un texto de entrada de cualquier longitud a un hash o compendio. Son funciones unidireccionales (irreversibles) de cálculo sencillo.



Texto de entrada (M) de longitud variable.

$M \rightarrow H(M)$ siendo $H(M)$ de longitud fija (256 o 512 bits)

Imposible obtener M a partir de su resumen $H(M)$

Invulnerables/Robustos a ataques de colisión. Dado M es imposible encontrar $M' / M \neq M'$ y $H(M) = H(M')$; es decir, es imposible encontrar un texto diferente con el mismo hash.

Ejemplos de funciones HASH: MD5, SHA-1, SHA-512

Las funciones Hash se usan para garantizar integridad + autenticación.

Integridad: Si tengo un texto, el sistema me garantiza que cualquier alteración al transmitirse se notifique.

Le añado algo al mensaje para garantizar su integridad (concateno una clave secreta que tengo con el destino y el texto). Cualquier alteración de M, intencionada o no, se detecta gracias a que el destinatario hace el hash de $H(K|M')$ y lo compara.

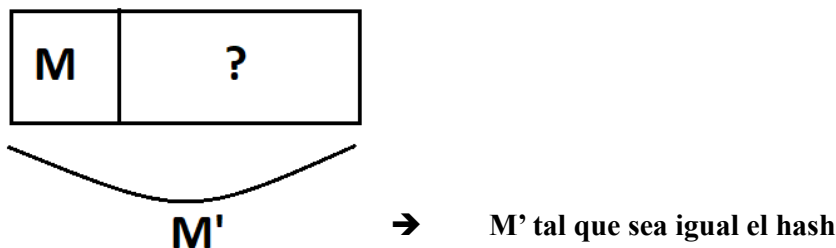
$$M + H(K|M) \xrightarrow{\quad\quad\quad} M' + H(K|M)$$

$$\downarrow$$

$$V$$

$$\text{¿}H(K|M') = H(K|M)\text{?}$$

Hash Message Authentication Code (HMAC): $M + H(K|M)$ pero para evitar ataques de extensión se usa $M + H(K | H(K|M))$.: Se les concatenan más cosas para evitar ataques de extensión



MD5 (“Message Digest 5”)

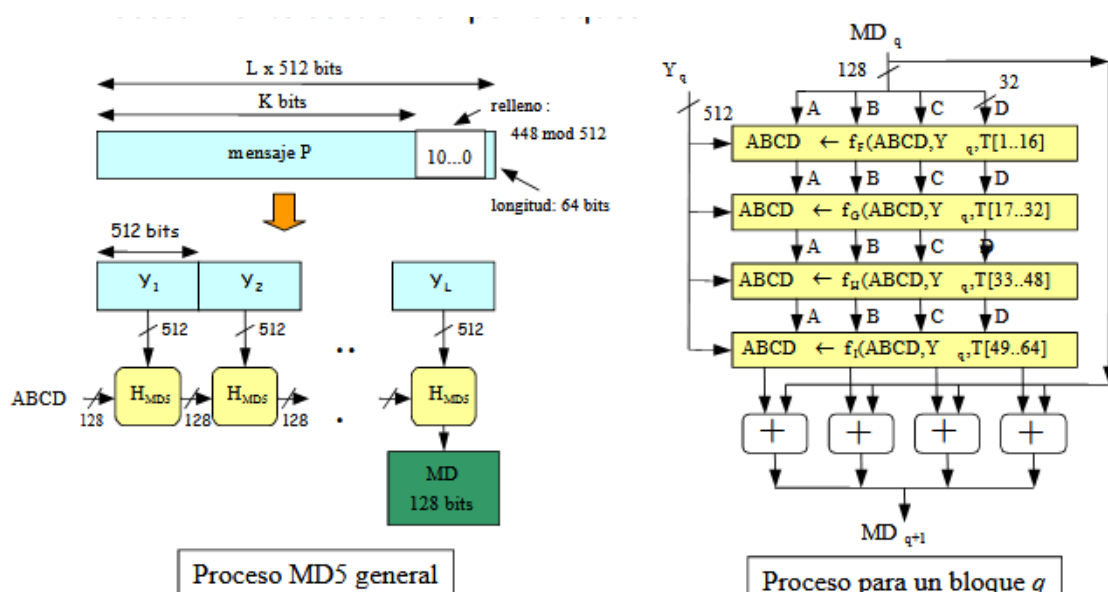
Proceso (resumen de 128 bits)

Relleno 100..0 de longitud máxima 448 bits

Adición de campo de longitud de 64 bits

División del mensaje en bloques de 512 bits

Procesamiento secuencial por bloques



SHA-1 ("Secure Hash Algorithm 1")

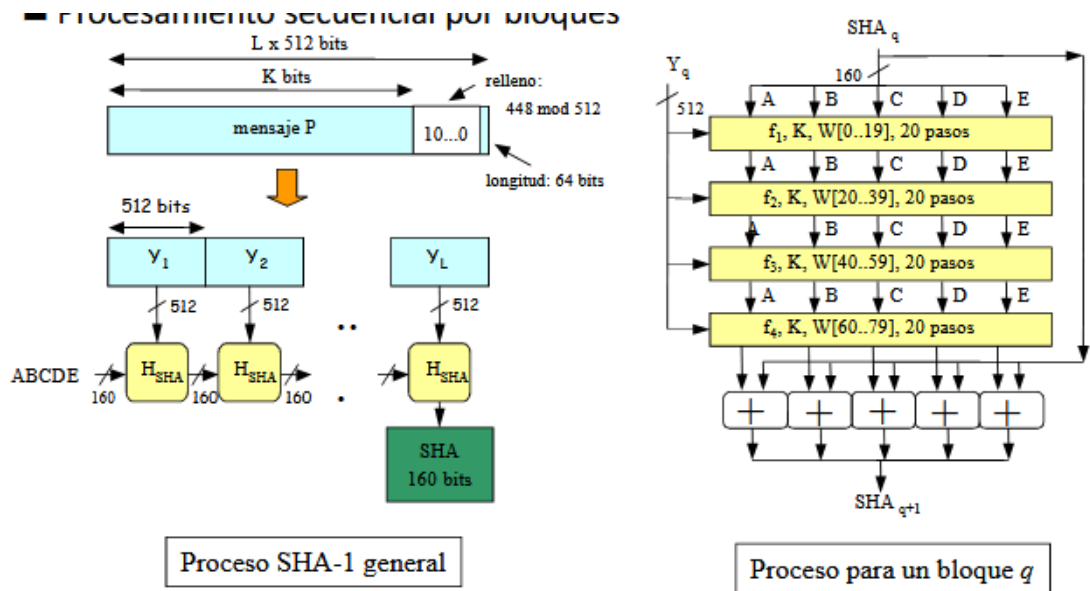
Proceso (resumen de 160 bits)

Relleno de 100..0 de longitud máxima 448 bits

Adición de campo de longitud de 64 bits

División del mensaje en bloques de 512 bits

Procesamiento secuencial por bloques



4.6. FIRMA DIGITAL

Objetivos de la firma digital:

El receptor pueda autenticar al emisor (el firmante).

No hay repudio (irrenunciabilidad): Una vez que firmo no puedo decir que no he firmado.

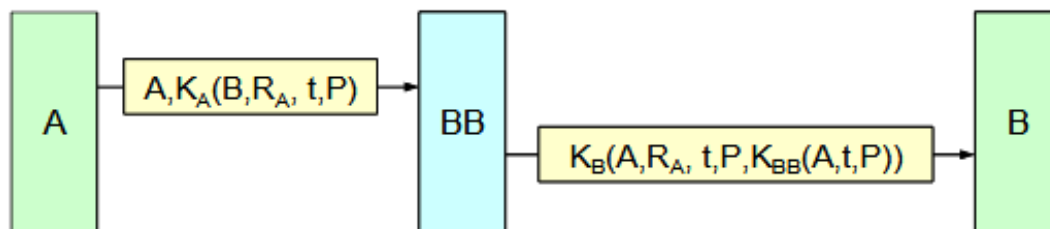
El emisor (el firmante) tenga garantías de no falsificación (integridad) por parte del destinatario. Que no se altere lo que firmo por el destinatario.

Firma digital con clave secreta. Hipótesis: existe una entidad de confianza, responsable, no vulnerable: Big brother:

Cada entidad tiene una Ksecreta con el big brother y el big brother tiene una clave secreta que sólo él conoce.

Queremos cifrar un texto plano P

t → nonce | R_A → random



¿El emisor tiene garantía de que se envía la información de forma segura? ¿Se garantizan los 3 objetivos? Si

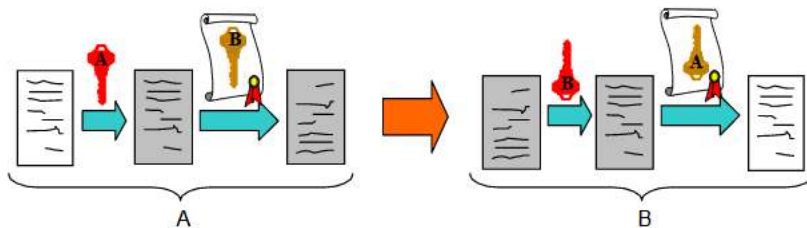
CON CLAVE ASIMÉTRICA. DOBLE CIFRADO:

Uno para proporcionar privacidad, con K_{PUB_B}

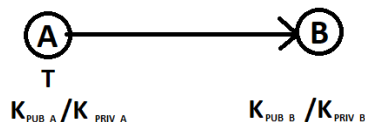
Otro, previo, para autenticación, con K_{PRIV_A}

Para firmar, enviar $K_{PUB_B}(K_{PRIV_A}(T)) \rightarrow$

En el receptor $K_{PUB_A}(K_{PRIV_B}(K_{PUB_B}(K_{PRIV_A}(T)))) = T$



A y B tienen sus claves públicas y privadas. A va a firmar a B un texto T



$K_{pub_B}(K_{priv_A}(T)) \rightarrow$ recibe $K_{pub_B}(K_{priv_A}(T))$
y hace $K_{pub_A}(K_{priv_B}(K_{pub_B}(K_{priv_A}(T)))) = T$

¿Cumple los 3 objetivos?

- ¿Autenticado A? Si, por la K_{priv_A} . Necesitamos que A esté asociado a su K_{pub} . ($A \rightarrow K_{PUB_A}$)
Alguien tiene que garantizarme esto $\rightarrow$ certificado digital. Se trata de cifrar por una autoridad (policía) la asociación. $\rightarrow K_{PRIV_POLICIA}(A \rightarrow K_{PUB_A})$
- ¿No repudio de A? Si. Si A niega que lo ha firmado, el destinatario coge lo recibido y lo lleva a la policía.
B descifra K_{PUB_B} y mira $K_{PRIV_A}(T) \rightarrow$ prueba de que A ha firmado ese texto porque B no lo puede hacer.
- Integridad: Si alguien ha cambiado el texto en el otro lado, voy a la policía y le piden al otro extremo lo que le ha llegado y mira. $K_{PRIV_A}(T) \rightarrow$ prueba de que A ha firmado ese texto.

Las tres propiedades se garantizan gracias al certificado digital y al doble cifrado.

Debilidad: para garantizar el no repudio, se necesita garantizar la asociación fehaciente e indisoluble de la “identidad A” con su “clave pública K_{PUB_A} ” ($A \leftrightarrow K_{PUB_A}$). Esto se consigue con un certificado digital.

Pero hay una vulnerabilidad: No impide/No tiene mecanismos para el ataque de denuncia falsa de robo de clave. Es decir, digo que alguien me ha robado la clave. La seguridad llega hasta aquí. Estas denuncias son imposibles de vencer.

4.7. CERTIFICADOS DIGITALES

Para garantizar que la asociación $A \leftrightarrow K_{PUB_A}$ (identidad $\leftrightarrow$ clave pública) sea fehaciente, necesitamos una autoridad que la cifre: El certificado digital.

$$K_{PUB_AUT} (K_{PRIV_AUT} (A, K_{PUB_A}))$$

CERTIFICADO DIGITAL

Se necesita la intervención de una Autoridad de Certificación (AC):

- El usuario obtiene sus claves pública y privada.
- Éste envía una solicitud, firmada digitalmente, a la AC indicando su identidad y su clave pública.
- AC comprueba la firma y emite el certificado solicitado:
 - Identidad de AC, identidad del usuario, clave pública del usuario y otros datos como, por ejemplo, el período de validez del certificado.
 - Todo ello se firma digitalmente con la clave privada de AC con objeto de que el certificado no pueda falsificarse.

En la administración hay una serie de estándares/formato de certificados. En concreto, la norma x509, reconocida en la administración española.

AC reconocidas:

- ACE
- CAMERFIRMA
- CERES
- VeriSign

CAMPOS DE UN CERTIFICADO X509

Field	Explanation
Version	Número de versión de X509
Número de serie	Identificador único usado por la AC
Firma	Firma de certificado (autoridad)
Editor	Nombre del CA definido por X509
Periodo de validez	Periodo de tiempo en el que el certificado es válido. Te vincula a tu clave pública durante un tiempo
Sujeto	La entidad cuya clave pública está siendo certificada
Clave pública	La llave pública y los algoritmos que usa

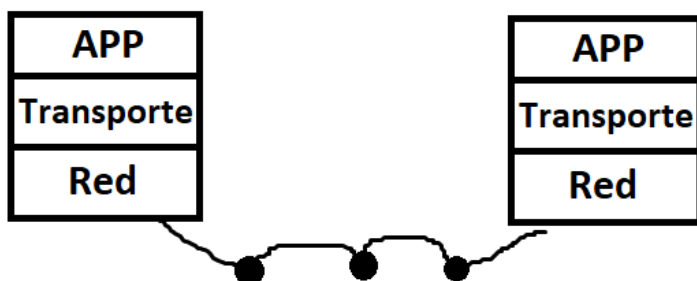
Certificate:
Data:
Version: 1 (0x0)
Serial Number: 7829 (0x1e95)
Signature Algorithm: md5WithRSAEncryption
Issuer: C=ZA, ST=Western Cape, L=Cape Town, O=Thawte Consulting cc, OU=Certification Services Division, CN=Thawte Server CA/Email=server-certs@thawte.com
Validity
Not Before: Jul 9 16:04:02 1998 GMT
Not After : Jul 9 16:04:02 1999 GMT
Subject: C=US, ST=Maryland, L=Pasadena, O=Srent Baccala, OU=FreeSoft, CN=www.freessoft.org/Email=baccala@freessoft.org
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
RSA Public Key: (1024 bit)
Modulus (1024 bit):
00:b4:31:98:0a:c4:bc:62:c1:88:aa:dc:b0:c8:bb:
33:35:19:d5:0c:64:b9:3d:41:b2:96:fc:f3:21:e1:
66:36:d0:8e:56:12:44:ba:75:eb:e8:1c:9c:5b:66:
70:23:52:14:c9:ec:4f:91:51:70:29:de:53:85:17:
16:94:6e:ee:f4:d5:6f:d8:ca:b3:47:5e:1b:0c:7b:
c5:cc:2b:6b:c1:90:c3:16:31:0d:bf:7a:c7:47:77:
8f:a0:21:c7:4c:d0:16:65:00:c1:0f:d7:b8:80:e3:
d2:75:6b:c1:ea:9e:5c:ea:7d:c1:a1:10:bc:b8:
e8:35:1c:9e:27:52:7e:41:8f
Exponent: 65537 (0x10001)
Signature Algorithm: md5WithRSAEncryption
92:5f:8f:5f:c5:af:bf:0a:ab:a5:6d:fb:24:5f:b6:59:5d:9d:
92:2e:4a:1b:8b:ac:7d:99:17:5d:cd:19:f6:ad:ef:63:2f:92:
ab:2f:4b:c7:0a:13:90:ee:2c:0e:43:03:be:f6:ea:8e:9c:67:
d0:a2:40:03:f7:ef:6a:15:09:79:a9:46:ed:b7:16:1b:41:72:
0d:19:aa:ad:dd:9a:df:ab:97:50:65:f5:5e:85:a6:ef:19:d1:
5a:de:9d:ea:63:cd:cb:cc:6d:5d:01:85:b5:6d:c8:f2:d9:f7:
8f:0e:fc:ba:1f:34:a9:96:6e:6c:cf:f2:ef:9b:bf:de:b5:22:
68:9f

4.8. PROTOCOLOS SEGUROS: IMPLEMENTACIÓN DE MECANISMOS DE SEGURIDAD

¿Cómo se usa todo lo que hemos visto?

En cuanto a seguridad, tenemos 2 tipos:

- Seguridad perimetral: Limitar el tráfico para entrar o salir a través de las premisas fijadas. Se hace con:
 - Firewall: una serie de reglas que se activan
 - Sistema de detección de intrusiones (IDS) y de respuesta (IRS): Mecanismo que inspecciona el tráfico y en base a un repertorio de reglas o a un mecanismo de entrenamiento, son capaces de detectar anomalías o patrones extraños en el tráfico y reaccionar/responder a estos.
- Seguridad en Protocolos (¿Dónde poner la seguridad?):
¿En qué parte de la arquitectura debemos poner la seguridad? En todas las capas.

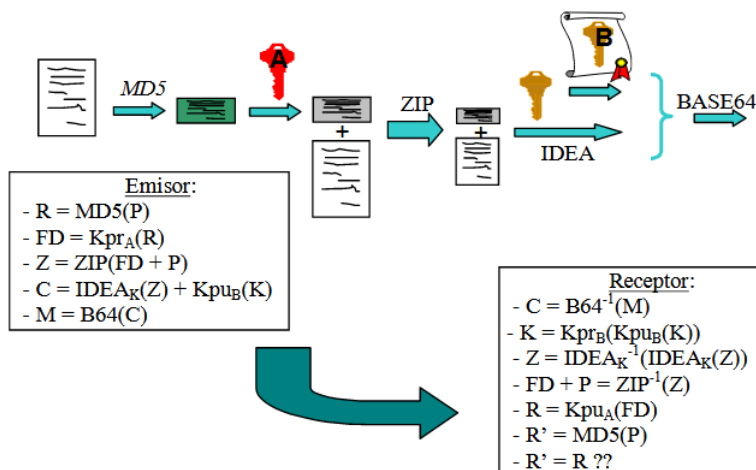


¿Qué pasa si a nivel de aplicación hago una conexión MUY SEGURA? ¿Tiene sentido poner seguridad en las capas de abajo? SI ¿Y al revés? TAMBIEN.

La seguridad tiene que estar en las 3 capas. ¿Cómo lo hacemos?:

- CAPA DE APLICACIÓN:
 - Pretty Good Privacy (PGP): Framework/Estándar/Iniciativa para enviar y recibir correo seguro. Ahora es opensource.

Cogemos el mensaje original y se hace un hash con MD5. Eso se cifra con la clave privada. El hash cifrado junto con el texto plano original se comprime. A esto se le cifra con un algoritmo de clave secreta (IDEA). Envía la clave secreta cifrada con la clave pública del destino al propio destino (Esto es porque la $K_{PUB/PRIV}$ se usan para cifrar cosas pequeñas <p.e la clave secreta>). Todo esto lo cifra en base 64.



¿Están garantizados los 3 objetivos?

- ✓ ¿Autenticado A? Si, por la K_{priv_A} . Necesitamos que A esté asociado a su K_{pub} . ($A \rightarrow K_{pub_A}$)
 - Alguien tiene que garantizarme esto $\rightarrow$ certificado digital. Se trata de cifrar por una autoridad (policía) la asociación. $\rightarrow K_{priv_POLICIA}$ ($A \rightarrow K_{pub_A}$)
- ✓ ¿No repudio de A? Si. Si A niega que lo ha firmado, el destinatario coge lo recibido y lo lleva a la policía. B descifra K_{pub_B} y mira $K_{priv_A}(T) \rightarrow$ prueba de que A ha firmado ese texto porque B no lo puede hacer.
- ✓ Integridad: Si alguien ha cambiado el texto en el otro lado, voy a la policía y le piden al otro extremo lo que le ha llegado y mira. $K_{priv_A}(T) \rightarrow$ prueba de que A ha firmado ese texto.

- Secure Shell (SSH): acceso remoto seguro. Es una mejora de telnet
- DNSSEC: DNS tiene una versión segura definiendo los registros con unas claves para firmar. Es un DNS con RR (RRSIG, DNSKEY) para firmar las respuestas

○ CAPA DE TRANSPORTE:

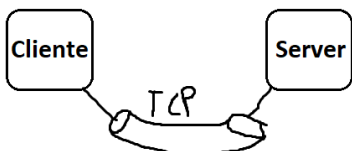
- Transport Secure Layer (TSL) (antes SSL) $\rightarrow$ Versión segura de TCP.
- TLS = Handshake (negociar) + Record Protocol (operación).
- TLS $\rightarrow$ Confidencialidad ($K_{secreta}$ negociada) + Autenticación (para el server por defecto con $K_{publica}$) + integridad (con HMAC)
 - HTTPS (HTTP = TLS), IMAPS, SSL-POP, VPN.
 - Es un ecosistema de protocolos. Son 4 protocolos:

IP	TCP	TLS	HTTP
----	-----	-----	------

$\rightarrow$ La aplicación (HTTP) la encapsulo en TLS y a su vez esto se encapsula en TCP.

El SSL Record Protocol encapsula los protocolos y ofrece un canal seguro con privacidad, autenticación e integridad.

Antes, en toda sesión que vaya a usar TLS, se usa SSL HandShake protocol.



Vamos a usar un intermediario TLS:



- Para que extremo a extremo hagan una negociación del algoritmo del cifrado (para dar flexibilidad), de la función hash....
- Autentica al servidor usando su certificado. El cliente autentifica al servidor enviando todo con el certificado. El cliente autentica al servidor con X.509
 - $K_{priv_AUT}(\text{Server}, K_{pub_SERVER})$
- El cliente genera claves de sesión:
 - Aleatorias cifrada con K_{pub_SERVER} (como en PGP)
 - Diffie-Hellman

SSL Handshake Protocol	SSL Change Cipher Spec Protocol	SSL Alert Protocol	HTTP
SSL Record Protocol			
TCP			
IP			

ASSERT PROTOCOL: Informa sobre errores en la sesión

Change Cipher Spec Protocol: Para notificar cambios en el cifrado

SSL RECORD PROTOCOL

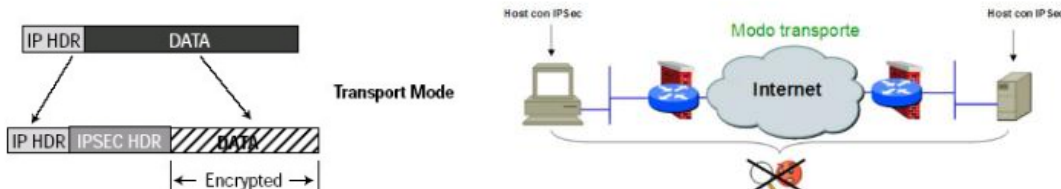
Los mensajes de la aplicación se trocean. Cada trozo se comprime. Se genera un hash (HMAC) y el compendio se combina con el mensaje. Se encripta. Se transmite.

Con SSL se autentica al servidor, no al cliente.

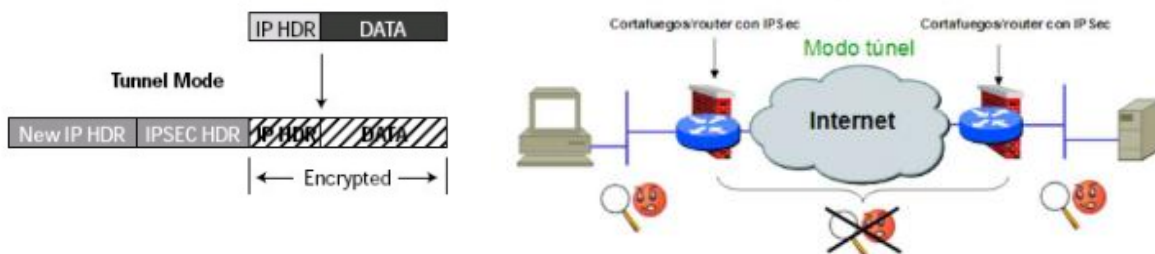
- CAPA DE RED: IPSec (VPN): Su objetivo es garantizar autenticación, integridad y (opcionalmente) privacidad a nivel IP. Son tres procedimientos:
 1. Establecimiento de una asociación de seguridad
 - Protocolo IKE (Internet Key Exchange)
 - Objetivo: establecimiento de la clave secreta (/de sesión) entre dos entidades IP. Se hace mediante Diffie-Hellman. Establecer un vínculo entre dos entidades mediante una clave de sesión.
 - Antes se necesita autenticación previa con el certificado digital del cliente y servidor. Esto es para evitar el ataque de persona en medio (Man in the middle).
 - Es simplex: La asociación de seguridad tiene un único sentido. Permite establecer una clave secreta en una dirección → asociación en un sentido (por ejemplo del cliente al servidor sólo, o al revés).
 - Se identifica con la IP origen + Security Parameter Index (32 bits).
 - Vulnera el carácter NO orientado a conexión de IP.
 2. Garantizar la autenticación e integridad de los datos:
 - Protocolo de “Cabeceras de autenticación”. Es como una versión ligera del IPSec. Parte de que ya tenemos la clave secreta del 1º.
 3. (Opcional) Garantizar la autenticación e integridad y privacidad de los datos:
 - Protocolo de “Encapsulado de seguridad de la carga”. Para el punto anterior se puede utilizar esto. Es una versión más completa y robusta que, además, garantiza la seguridad (cifra).

En IPSec hay dos modos de operación:

- ★ Modo Transporte: La asociación se hace extremo a extremo entre el host origen y host destino. Garantizar extremo a extremo la sesión. Todo va cifrado extremo a extremo (del host origen al host destino) → Las IPs son de los hosts terminales.



- ★ Modo túnel: La asociación se hace entre dos routers intermediarios. La parte segura es la de Internet. Se crea una nueva cabecera, se mete la cabecera de ese IPSEC para integridad y ¿autenticación?



VPN → Establecer una conexión con IPSec

